LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN 2

MODUL 4

“PROSEDUR”

DISUSUN OLEH:

DYAH IMANSARI

109082500130

S1IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

1. Definisi Prosedure

Prosedur adalah potongan beberapa instruksi program yang dianggap sebagai suatu instruksi baru. Tujuannya adalah untuk mengurangi kerumitan dari kode program yang kompleks pada suatu program yang besar.

Suatu subprogram dikatakan prosedur apabila memenuhi dua syarat utama:

- Tidak ada deklarasi tipe nilai yang dikembalikan (*return type*).
- Tidak terdapat kata kunci return dalam badan subprogram.

Penulisan nama prosedur sebaiknya berbentuk kata kerja (Contoh: cetak, hitungRerata, cariNilai).

2. Deklarasi Prosedure

Prosedur harus dideklarasikan di luar blok program utama (func main).

- **Deklarasi:** Ditulis di luar blok program utama (func main). Bisa diletakkan sebelum atau sesudah blok program utama.
- **Pemanggilan:** Prosedur hanya akan dieksekusi apabila dipanggil oleh program utama (baik secara langsung maupun tidak langsung melalui subprogram lain). Cara memanggilnya adalah dengan menuliskan nama prosedur beserta parameter/argumen yang diminta.

3. Parameter Formal dan Parameter Aktual

Subprogram berkomunikasi dengan pemanggilnya melalui parameter:

- **Parameter Formal:** Parameter yang ditulis pada saat deklarasi subprogram. Berfungsi sebagai petunjuk argumen apa saja yang diperlukan.
- **Parameter Aktual:** Argumen (berupa variabel atau nilai langsung) yang digunakan pada saat pemanggilan subprogram. Jumlah dan tipe datanya harus mengikuti parameter formal.

4. Mekanisme Pengiriman Parameter

Berdasarkan alokasi memorinya, parameter dikelompokkan menjadi dua:

4.1. Pass by Value

- Nilai pada parameter aktual disalin ke variabel lokal (parameter formal).
- Alokasi alamat memori antara parameter aktual dan formal berbeda.

- Subprogram tidak dapat mengembalikan informasinya ke pemanggil melalui parameter aktual karena pemanggil tidak dapat mengakses memori subprogram.

4.2. Pass by Reference (Pointer)

- Parameter formal berperan sebagai pointer yang menyimpan alamat memori dari parameter aktual.
- Perubahan nilai pada parameter formal akan berdampak langsung pada parameter aktual.
- Dalam bahasa Go, ditandai dengan identifier asterik (*) sebelum tipe data pada parameter formal.
- Mekanisme ini sangat disarankan untuk prosedur agar prosedur seolah-olah bisa mengirimkan nilai kepada pemanggil.

Dalam notasi pseudocode, *pass by value* biasanya menggunakan kata kunci in, sedangkan *pass by reference* menggunakan kata kunci in/out.

LATIHAN

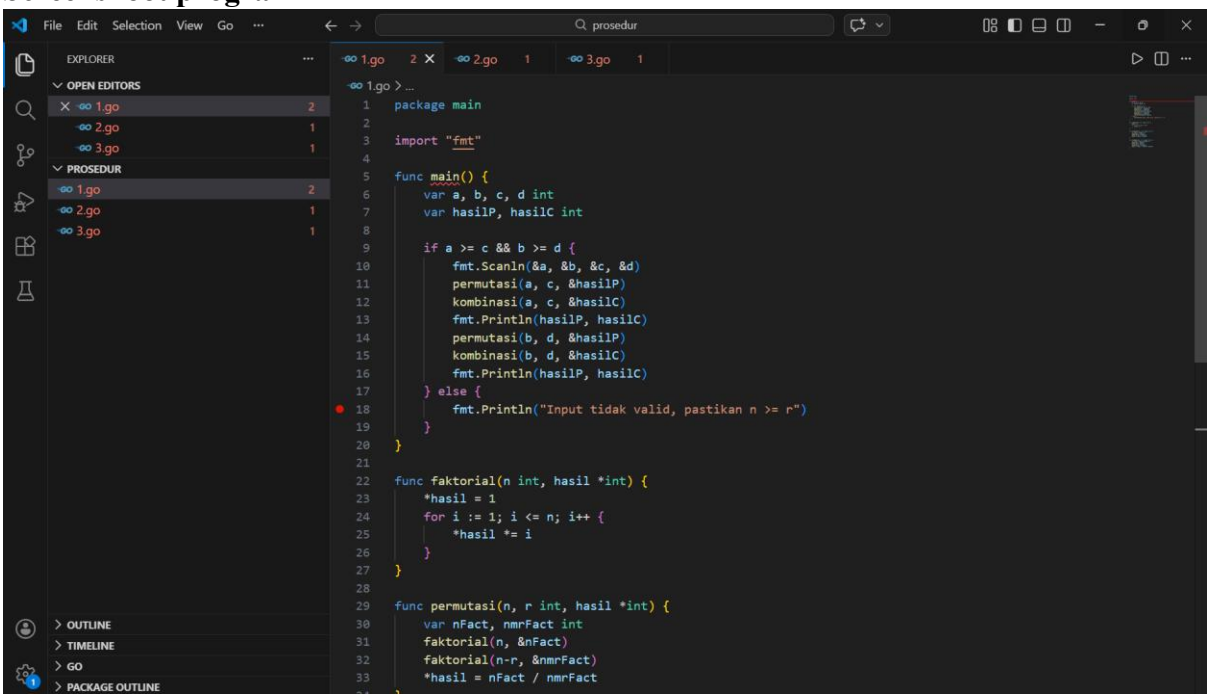
1. **Latihan 1**

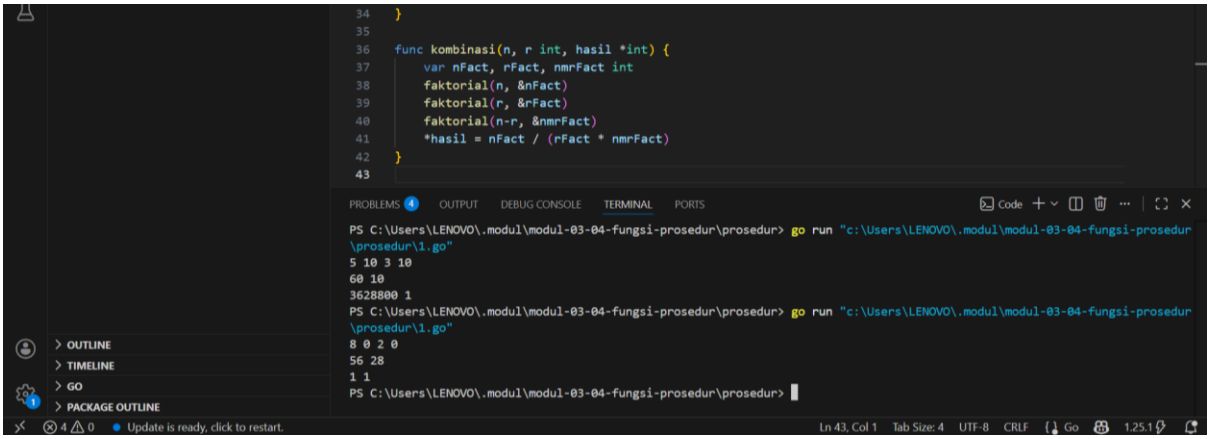
Source Code

```
package main

import "fmt"

func main() {
    var a, b, c, d int
    var hasilP, hasilC int

    if a >= c && b >= d {
        fmt.Scanln(&a, &b, &c, &d)
        permutasi(a, c, &hasilP)
        kombinasi(a, c, &hasilC)
        fmt.Println(hasilP, hasilC)
        permutasi(b, d, &hasilP)
        kombinasi(b, d, &hasilC)
        fmt.Println(hasilP, hasilC)
    } else {
        fmt.Println("Input tidak valid, pastikan n >=
r")
    }
}

func faktorial(n int, hasil *int) {
    *hasil = 1
    for i := 1; i <= n; i++ {
        *hasil *= i
    }
}

func permutasi(n, r int, hasil *int) {
    var nFact, nmrFact int
    faktorial(n, &nFact)
    faktorial(n-r, &nmrFact)
    *hasil = nFact / nmrFact
```

```
}

func kombinasi(n, r int, hasil *int) {
    var nFact, rFact, nmrFact int
    faktorial(n, &nFact)
    faktorial(r, &rFact)
    faktorial(n-r, &nmrFact)
    *hasil = nFact / (rFact * nmrFact)
}
```

Screenshoot program





Deskripsi program

Program ini dimulai dengan mengambil 4 angka dari pengguna, lalu mengecek apakah angka-angka tersebut bisa dihitung (angka pertama harus lebih besar atau sama dengan angka kedua). Langkah ini sangat penting agar program tidak mengalami kesalahan saat menghitung faktorial. Program ini menggunakan sistem "prosedur" dan "pointer", yang fungsinya agar subprogram bisa langsung mengubah isi variabel di program utama tanpa harus mengirim balik nilai secara manual.

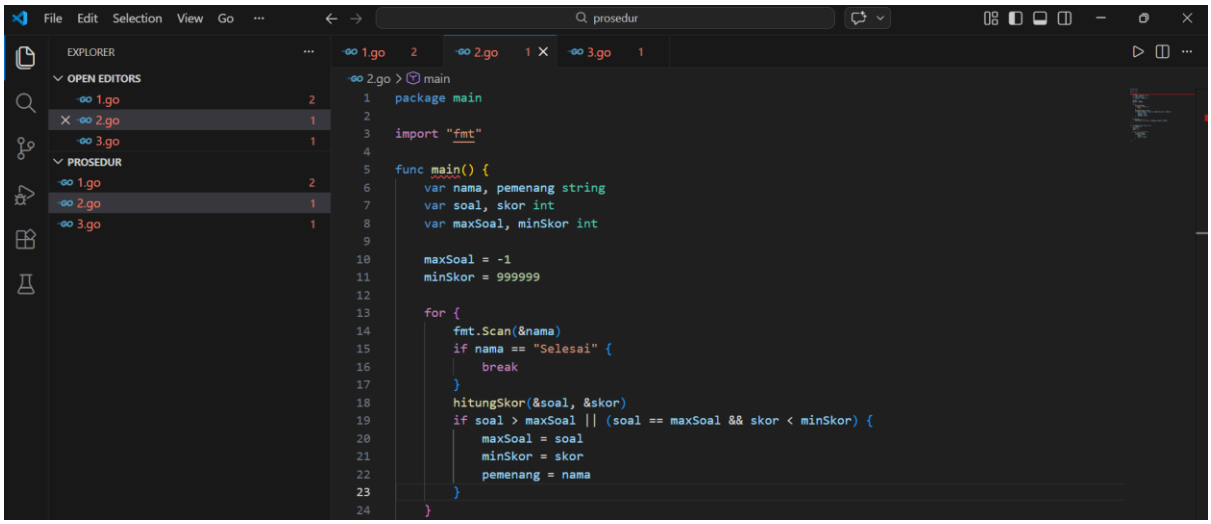
Untuk cara kerjanya, program akan menjalankan prosedur faktorial terlebih dahulu sebagai dasar perhitungan. Setelah itu, prosedur permutasi dan kombinasi akan memanggil hasil faktorial tersebut untuk diproses sesuai rumusnya masing-masing. Hasil dari pasangan angka pertama akan langsung dicetak ke layar, kemudian program mengulangi langkah yang sama untuk pasangan angka kedua.

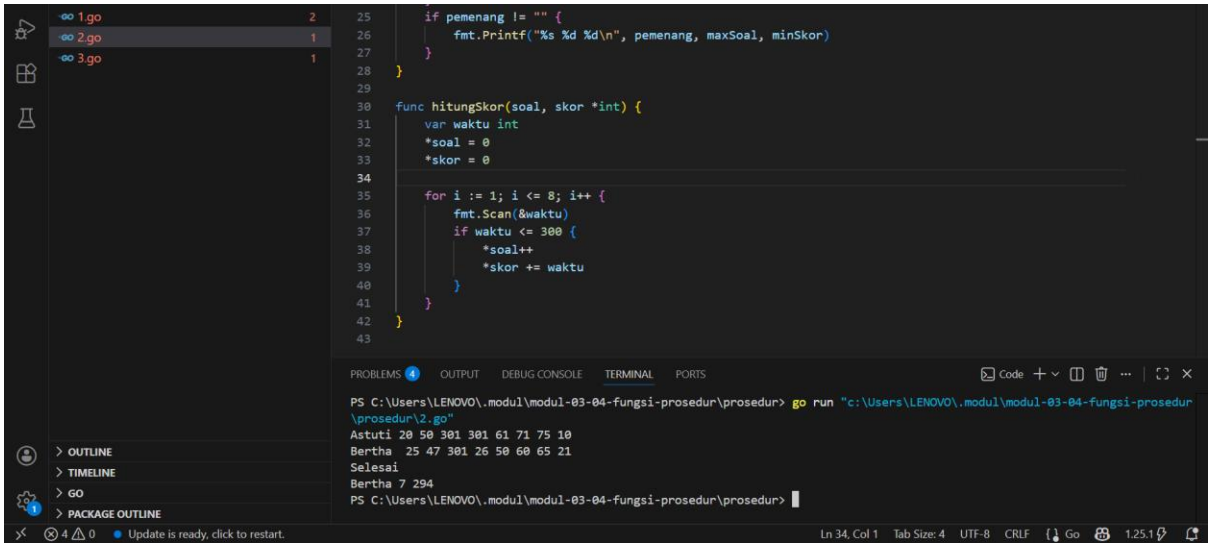
2. **Latihan 2**
   Source Code

```
package main

import "fmt"

func main() {
    var nama, pemenang string
    var soal, skor int
    var maxSoal, minSkor int

    maxSoal = -1
    minSkor = 999999

    for {
        fmt.Scan(&nama)
        if nama == "Selesai" {
            break
        }
        hitungSkor(&soal, &skor)
        if soal > maxSoal || (soal == maxSoal && skor <
minSkor) {
            maxSoal = soal
            minSkor = skor
            pemenang = nama
        }
    }
```

```
        if pemenang != "" {
            fmt.Printf("%s %d %d\n", pemenang, maxSoal,
    minSkor)
        }
    }

    func hitungSkor(soal, skor *int) {
        var waktu int
        *soal = 0
        *skor = 0

        for i := 1; i <= 8; i++ {
            fmt.Scan(&waktu)
            if waktu <= 300 {
                *soal++
                *skor += waktu
            }
        }
    }
```

Screenshoot program





Deskripsi program

Program ini dirancang untuk menentukan pemenang kompetisi berdasarkan jumlah soal terbanyak dan waktu pengerjaan tercepat. Di awal, program menyiapkan variabel untuk mencatat skor terbaik dan nama pemenang, lalu melakukan pengulangan untuk membaca data peserta satu per satu sampai muncul kata "Selesai". Program ini menggunakan prosedur hitungSkor dengan sistem pointer (*int), yang fungsinya agar data jumlah soal dan total waktu yang dihitung di dalam prosedur bisa langsung tersimpan ke variabel di program utama tanpa perlu nilai balik (return).

Cara kerjanya, setiap kali nama peserta dibaca, prosedur hitungSkor akan mengambil 8 data waktu pengerjaan. Jika waktu pengerjaan suatu soal tidak lebih dari 300 menit, maka jumlah soal yang diselesaikan akan bertambah dan waktunya akan diakumulasikan ke total skor. Setelah prosedur selesai, program utama akan mengecek apakah peserta tersebut memiliki jumlah soal lebih banyak dari pemenang sebelumnya, atau jika jumlah soalnya sama, apakah waktunya lebih singkat. Jika memenuhi syarat, data peserta tersebut akan disimpan sebagai pemenang dan akhirnya nama pemenang beserta total skornya dicetak di akhir program.

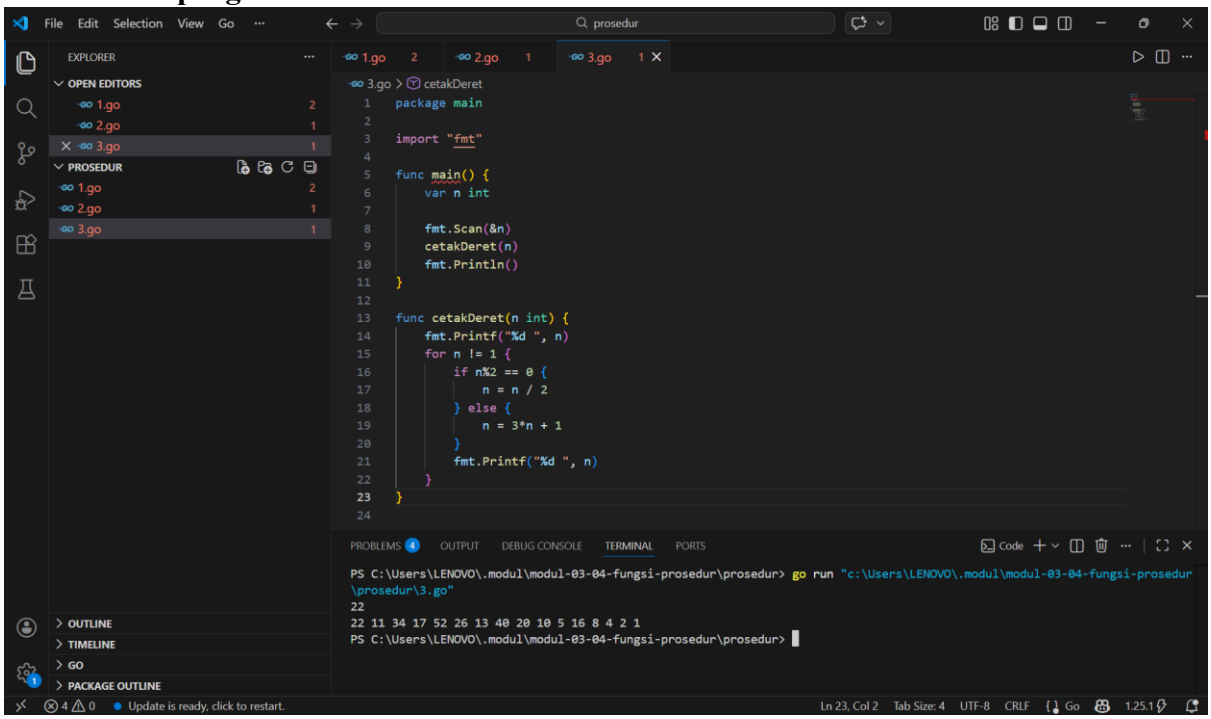
3. Latihan 3

Source Code

```
package main

import "fmt"

func main() {
    var n int

    fmt.Scan(&n)
    cetakDeret(n)
    fmt.Println()
```

```
}

func cetakDeret(n int) {
    fmt.Printf("%d ", n)
    for n != 1 {
        if n%2 == 0 {
            n = n / 2
        } else {
            n = 3*n + 1
        }
        fmt.Printf("%d ", n)
    }
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mencetak deret bilangan yang mengikuti aturan matematika tertentu berdasarkan satu angka awal yang dimasukkan oleh pengguna. Program ini menggunakan prosedur bernama cetakDeret yang menerima satu parameter input berupa nilai awal. Karena hanya mencetak angka dan tidak perlu mengirimkan hasil perhitungan kembali ke program utama, program ini dibuat dalam bentuk prosedur tanpa nilai kembalian (return value).

Cara kerjanya, setelah pengguna memasukkan angka, program akan mengecek apakah angka tersebut genap atau ganjil secara berulang sampai angka terakhirnya bernilai 1. Jika angka saat itu genap, maka angka berikutnya adalah setengahnya (n/2), namun jika ganjil, angka berikutnya dihitung dengan rumus 3n+1.

DAFTAR PUSTAKA

Tim Penyusun. (2026). *Modul Praktikum Algoritma dan Pemrograman 2*. Bandung: Fakultas Informatika, Telkom University.